Vapor Stone RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Watchdog Module

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-09-12
Verified :	Eric Malo, Eng. System Engineer	2011-09-12
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011-09-12

Document No.	**201111544**	Revision	**0**
		Date (yyyy-mm-dd)	**2011-08-26**

Template 200610247-00R-A

Based on SDD Template S17-07 Rev.1

Revision List

Rev.	ECD#	Date	By	Description of changes
0	---	2011-08-26	James Kim	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and 1558-2004.

Table of Contents

REVISION LIST **2**

PREFACE **3**

TABLE OF CONTENTS **4**

LIST OF FIGURES **5**

LIST OF TABLES **5**

1 INTRODUCTION **6**

1.1 Purpose 6
1.2 Scope 6
1.3 Definitions and Acronyms 6
1.3.1 Definitions 6
1.3.2 Acronyms 6
1.4 Document Organization 7

2 REFERENCES **8**

3 SOFTWARE DESIGN **9**

3.1 Decomposition Description 9
3.1.1 Module Decomposition 9
3.1.1.1 WD Module Description 9
3.1.2 Concurrent Process Decomposition 9
3.1.2.1 Watchdog Task Description 9
3.1.3 Data Decomposition 9
3.1.3.1 Watchdog Data Description 9
3.2 Dependency Description 10
3.2.1 Inter-Module Dependencies 10
3.2.1.1 Watchdog Module 10
3.2.2 Inter-Process Dependencies 10
3.2.3 Data Dependencies 10
3.3 Interface Description 11
3.3.1 Module Interfaces 11
3.3.1.1 Watchdog Module External Interfaces 11
3.3.1.2 Watchdog Module Internal Interface 12
3.4 Detailed Design 13
3.4.1 Watchdog Module Detailed Design 13
3.4.1.1 WATCHDOG_Close Function 13
3.4.1.2 WATCHDOG_DeleteTask Function 13
3.4.1.3 WATCHDOG_GetName Function 13
3.4.1.4 WATCHDOG_GetVersion Function 14
3.4.1.5 WATCHDOG_Init Function 14
3.4.1.6 WATCHDOG_Open Function 14
3.4.1.7 WATCHDOG_StatusGet Function 14
3.4.1.8 WATCHDOG_Trig Function 15
3.4.1.9 watchdog_Task Function 16
3.4.2 Data Entities Detailed Design 16

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 10
Figure 3-2: WATCHDOG_DeleteTask Flow Chart 13
Figure 3-3: WATCHDOG_GetName Flow Chart 13
Figure 3-4: WATCHDOG_GetVersion Flow Chart 14
Figure 3-5: WATCHDOG_Init Flow Chart 14
Figure 3-6: WATCHDOG_StatusGet Flow Chart 14
Figure 3-7: WATCHDOG_Trig Flow Chart 15
Figure 3-8: watchdog_Task Flow Chart 16

List of Tables

Table 3-1: Watchdog Version 9
Table 3-2: Watchdog Name 9
Table 3-3: Watchdog State 9

1 INTRODUCTION

This document is the Software Design Description of the Watchdog (WD) Module developed by Vapor Stone Rail Systems.

The WD Module is a part of the Door Control Module (DCM) Application that manages the watchdog reset feature.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software components and implementation details required of the WD Module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.2 Scope

The WD Module is a software module that can be used by a DCM Application. It makes sure that all modules in the DCM Application won't freeze its operation.
This module interfaces with other modules and, it is not responsible for a door operation.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Control Module:	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.

1.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CPU	Central Processing Unit
DCM	Door Control Module
ECD	Engineering Change Document
IEEE	Institute of Electrical and Electronics Engineers
KB	Kilobytes
LED	Light Emitting Diode
LLC	Limited Liability Corporation
MHz	MegaHertz
ms	millisecond
NVRAM	Non-Volatile Random Access Memory
OS	Operating System
PTA	Portable Test Application
RAM	Random Access Memory
SCI	Software Configuration Items
SDD	Software Design Description
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
TS	Technical Specification
VSRS	Vapor Stone Rail Systems
WD	Watchdog

1.4 Document Organization

This SDD in divided into six sections and the following specific information is provided within each one:

- Section 1, Introduction states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interface for each entities.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependency between different modules.
- Tables contain data structures used by the WD Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Module Decomposition

3.1.1.1 WD Module Description

The WD Module performs following function:

- Manage the hardware watchdog reset feature

3.1.2 Concurrent Process Decomposition

3.1.2.1 Watchdog Task Description

The WD task loop runs every 250ms and verifies the amount of calls that all tasks have made during the watchdog configured period.

3.1.3 Data Decomposition

3.1.3.1 Watchdog Data Description

Table 3-1: Watchdog Version

```
static unsigned char ucWatchdog_VersionMajor = [Major];
static unsigned char ucWatchdog_VersionMinor = [Minor];
static unsigned char ucWatchdog_VersionCorrection = [Correction];
```

Table 3-2: Watchdog Name

```
static unsigned char *pucWatchdog_Name = "Watchdog";
```

Table 3-3: Watchdog State

```
enum Watchdog_State (UNKNOWN, INIT, OPEN, CLOSED, INVALID);
```

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

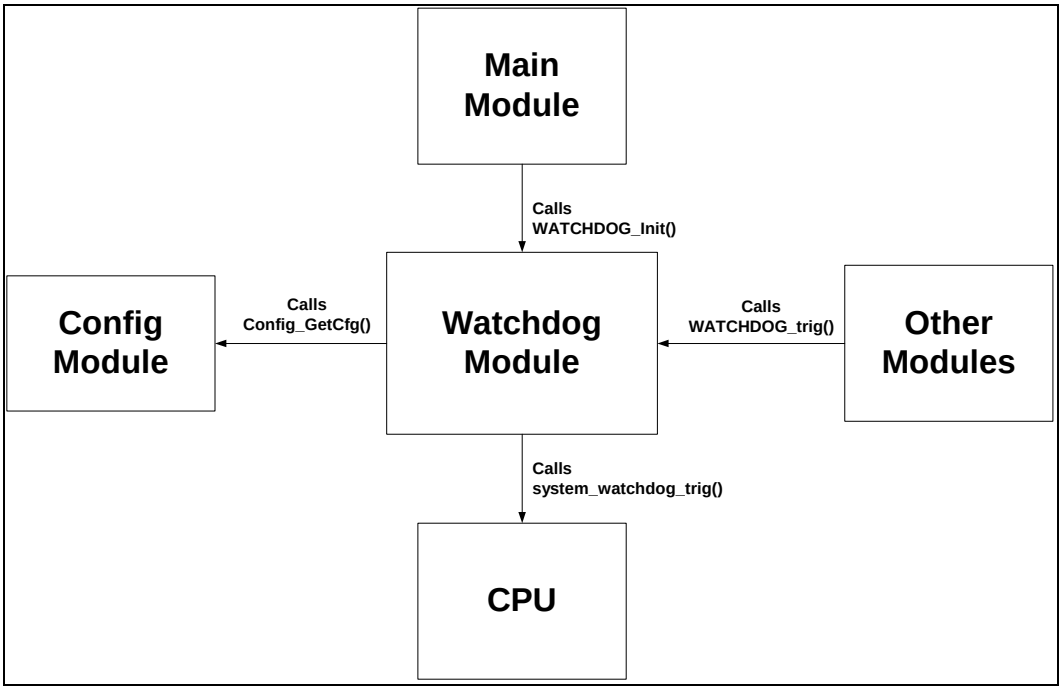

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 Watchdog Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the WD Module.

3.2.2 Inter-Process Dependencies

Not applicable.

3.2.3 Data Dependencies

See section 3.2.1 for data dependencies.

3.3 Interface Description

This section describes the functions of WD Module, listed in alphabetical order.

3.3.1 Module Interfaces

3.3.1.1 Watchdog Module External Interfaces

Before the other module uses the WATCHDOG_Trig, the Main module must first call the WATHDOG_Init function.

Function Name	void WATCHDOG_Close (void)
Description	This function closes module.
Input	None.
Output	None.
Return	None.

Function Name	void WATCHDOG_DeleteTask (void)
Description	This function deletes the watchdog task id created.
Input	None.
Output	None.
Return	None.

Function Name	const char* WATCHDOG_GetName (void)
Description	This function returns the pointer of the entity name.
Input	None.
Output	None.
Return	The watchdog name in string with null terminated.

Function Name	void WATCHDOG_GetVersion (int *pMajor, int *pMinor, int *pCorrection)
Description	This function returns the watchdog version.
Input	None.
Output	pMajor: The major version number. pMinor: The minor version number. pCorrection: The correction version number.
Return	None.

Function Name	int WATCHDOG_Init (unsigned char ucInitDelay, unsigned char ucWDPeriod)
Description	This function initializes the internal module variable and gets watchdog configuration information from the config module. It calls the Config_WdtGet() functions to get number of the tasks and expected watchdog trigger number for each task. At lat it creates the watchdog task. If the watchdog task creation fails, it stays in the loop for the watchdog reset.
Input	ucInitDelay: Initial delay to make sure all tasks are created in other modules. ucWDPeriod: Period that watchdog verifies the amount of task calls.
Output	None.
Return	This function returns 0 if the configuration is successful. Otherwise, it returns -1 for error.

Function Name	int WATCHDOG_Open (void)
Description	This function opens the module.
Input	None.
Output	None.
Return	The returned value is 0 if everything is successful, otherwise it returns -1 for an error.

Function Name	bit WATCHDOG_StatGet (void)
Description	This function returns the watchdog state.
Input	None.
Output	None.
Return	This function returns watchdog state as seen in Table 3-3.

Function Name	bit WATCHDOG_StatusGet (void)
Description	This function returns the watchdog status bit. This bit is set if the watchdog reset has occured.
Input	None.
Output	None.
Return	This function returns bit value either 0 for "no watchdog-reset" or 1 for "watchdog-reset".

Function Name	void WATCHDOG_Trig (unsigned char ucSource)
Description	This function is called from other module that has task. It increments the number of trigger for each task in the list and flags if corresponding task has triggered the watchdog with expected amount.
Input	ucSource: This variable is the identification of each task defined in the config.h interface.
Output	None.
Return	This function returns bit value either 0 for "no watchdog-reset" or 1 for "watchdog-reset".

3.3.1.2 Watchdog Module Internal Interface

Function Name	void watchdog_Task (void)
Description	This function is called from the WATCHDOG_Open() and runs the task. It verifies every configured period if all tasks in the list triggered the watchdog with an expected number. If anyone of the tasks in the list haven't triggered watchdog with an expected amount, it will stay in the loop for the watchdog reset.
Input	None.
Output	None.
Return	None.

3.4 Detailed Design

This section describes how the Watchdog Module works and what operation it performs.

3.4.1 Watchdog Module Detailed Design

The module implements functions to manage the hardware watchdog reset. The following sub-sections describe each function.

3.4.1.1 WATCHDOG_Close Function

This function closes the Watchdog_State.

3.4.1.2 WATCHDOG_DeleteTask Function

The **Error! Reference source not found.** describes the operation.

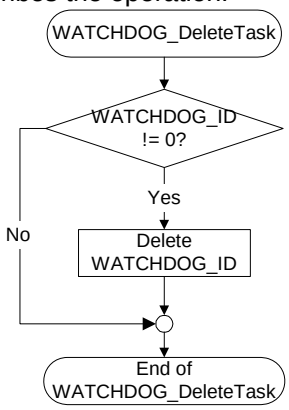

Figure 3-2: WATCHDOG_DeleteTask Flow Chart

3.4.1.3 WATCHDOG_GetName Function

The Figure 3-3 describes the operation.

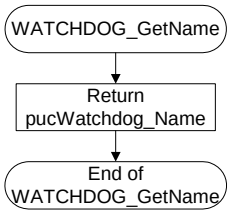

Figure 3-3: WATCHDOG_GetName Flow Chart

3.4.1.4 WATCHDOG_GetVersion Function

The Figure 3-4 describes the operation.

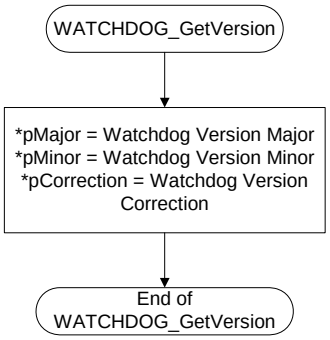

Figure 3-4: WATCHDOG_GetVersion Flow Chart

3.4.1.5 WATCHDOG_Init Function

The Figure 3-5 describes the operation.

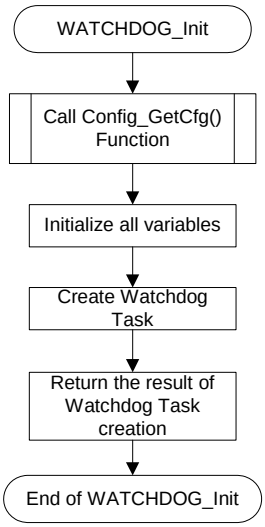

Figure 3-5: WATCHDOG_Init Flow Chart

3.4.1.6 WATCHDOG_Open Function

This function opens the Watchdog_State.

3.4.1.7 WATCHDOG_StatusGet Function

The Figure 3-6 describes the operation.

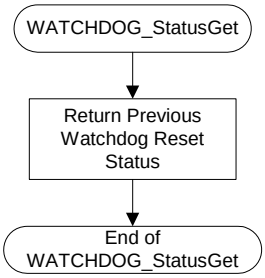

Figure 3-6: WATCHDOG_StatusGet Flow Chart

3.4.1.8 WATCHDOG_Trig Function

The Figure 3-7 describes the operation.

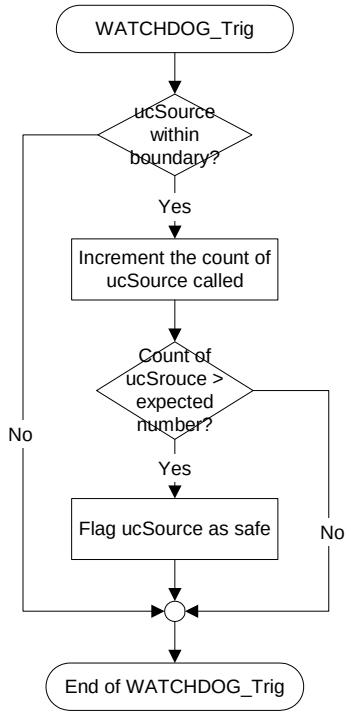

Figure 3-7: WATCHDOG_Trig Flow Chart

3.4.1.9 watchdog_Task Function

The Figure 3-8 describes the operation.

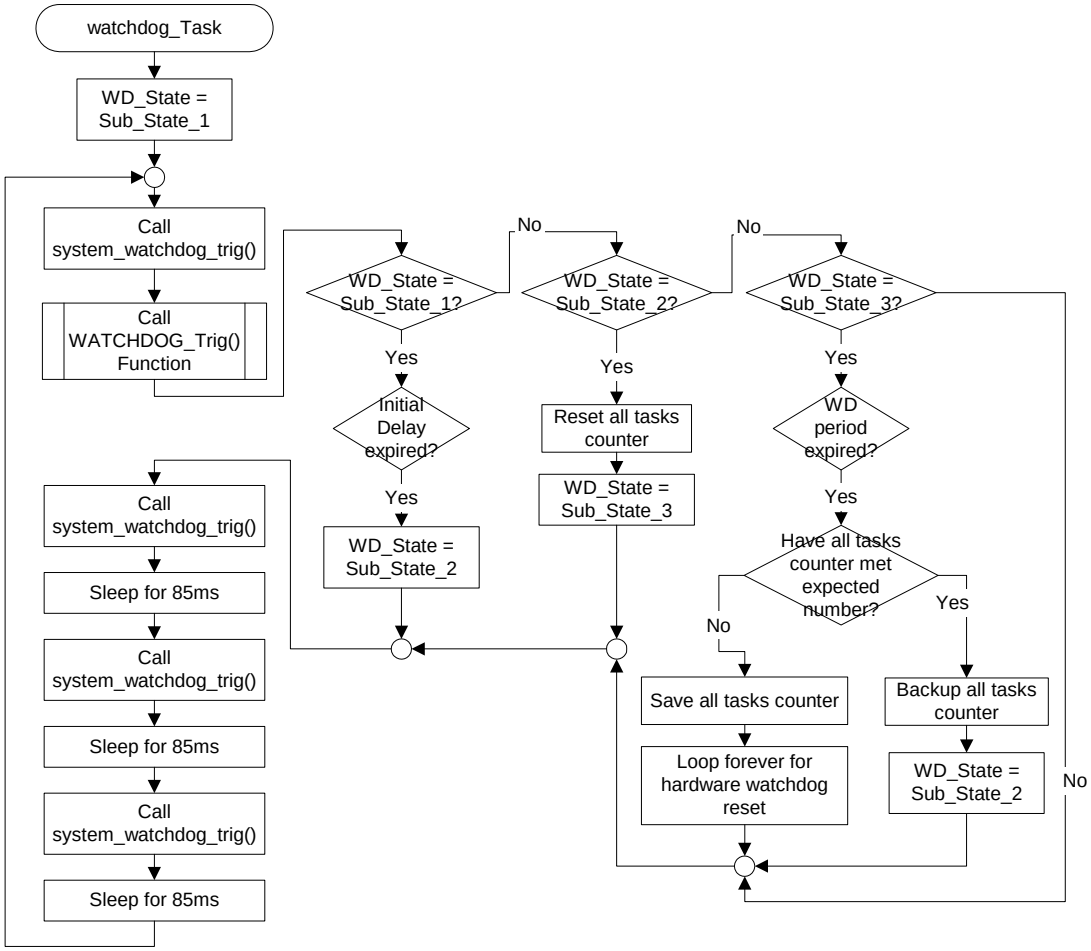

Figure 3-8: watchdog_Task Flow Chart

3.4.2 Data Entities Detailed Design

Not applicable.